END-TO-END LAST-MILE LOGISTICS ANALYTICS PIPELINE

Project Report

Python • PostgreSQL • dbt • Apache Airflow • Power BI

60,000	4,244,332	16 / 16	5-task DAG
Orders	Tracking events	dbt tests passed	Airflow success

Prepared by Md. Maidul Hassan Shanto

28 October 2026

Portfolio project using synthetic logistics data for learning and demonstration purposes.

Contents

Executive Summary

1. Project Context and Objectives

2. Dataset and Business Scope

3. Solution Architecture

4. Python ETL Pipeline

5. PostgreSQL and Analytics Layering

6. dbt Transformations and Data Quality

7. Apache Airflow Orchestration

8. Power BI Dashboard and Key Results

9. Validation, Reliability and Engineering Challenges

10. Limitations and Future Roadmap

11. Skills Demonstrated and Conclusion

Appendix A. Reproducible Run Sequence

Appendix B. Repository Structure

Report basis

This report documents the implementation and verified outputs produced during the project. It does not claim that the system is production-ready or that the synthetic dataset represents real company operations.

Executive Summary

This project implements an end-to-end last-mile logistics analytics workflow that converts operational CSV data into a validated PostgreSQL data layer, transforms it into analytics-ready models, tests data quality, orchestrates the workflow with Apache Airflow, and presents decision-oriented metrics in Power BI.

The project was designed as a portfolio-scale demonstration of both data engineering and business intelligence capabilities. The source dataset is synthetic and covers orders, tracking events, payments, returns, merchants, couriers and hubs. A Python ETL process performs schema validation, primary-key validation, missing-value profiling, bulk loading and source-to-database row-count verification. dbt then transforms the raw data into staging, intermediate, fact and KPI models. The final orchestration sequence runs Python ETL, dbt transformations and dbt tests through Airflow.

Verified outcome	Result	Meaning
Orders processed	**60,000**	Order-level operational volume
Tracking events processed	**4,244,332**	Large event-level dataset for delivery lifecycle analysis
dbt data-quality tests	**16 / 16 passed**	No failed configured tests in the final run
Airflow orchestration	**5 tasks succeeded**	start → Python ETL → dbt run → dbt test → end

Final dashboard headline metrics: 60K total orders, 75.82 SLA compliance, 44.15 hours average delivery TAT, and 92.42 delivery success rate. These figures are reported as displayed in the completed Power BI dashboard; the percentage metrics use the project's defined denominators.

1. Project Context and Objectives

Last-mile delivery operations generate multiple kinds of data: orders, hub assignments, courier activity, event histories, payments and returns. These datasets are useful only when they are consistently validated, integrated and transformed into reliable operational metrics. The project therefore focuses on the full path from raw operational files to management-level analytics.

Objectives

- Build a repeatable ingestion pipeline for multiple logistics source files.
- Validate schema structure and primary-key quality before loading data.
- Store source data in PostgreSQL using a clear raw layer.
- Use dbt to separate transformation logic from ingestion logic and create analytics-ready models.
- Apply automated data-quality tests to key business models and source identifiers.
- Use Airflow to enforce task dependency and orchestrate the complete pipeline.
- Create a concise Power BI dashboard that translates the engineered data into operational KPIs.
- Package the project so it can be explained in a CV, portfolio and technical interview.

Project positioning

The implementation uses production-style patterns such as environment-variable configuration, layered data modeling, automated tests and orchestration. However, it remains a portfolio project: scheduling, failure alerts, CI/CD, cloud deployment and incremental processing are intentionally left as future improvements.

2. Dataset and Business Scope

The project uses a synthetic last-mile logistics dataset organised into seven operational CSV files. The final successful ETL verification produced the following source and database row counts.

Entity	Rows	Business purpose
hubs	**12**	Delivery facilities and geographic routing points
merchants	**600**	Shippers generating orders
couriers	**220**	Delivery resources assigned to orders
orders	**60,000**	Core shipment and service-level records
tracking_events	**4,244,332**	Event history across the delivery lifecycle
payments	**60,000**	Shipping fee, COD, settlement and reconciliation data
returns	**2,930**	Return initiation, status, reason and cost data

Business dimensions represented: origin and destination hubs, destination city/zone, courier, merchant, service type, package category, payment type, order source and operational status. These dimensions support performance analysis by time, location, courier and order outcome.

3. Solution Architecture

The architecture deliberately separates ingestion, storage, transformation, testing, orchestration and visualization. This improves maintainability and makes the responsibilities of each tool explicit.

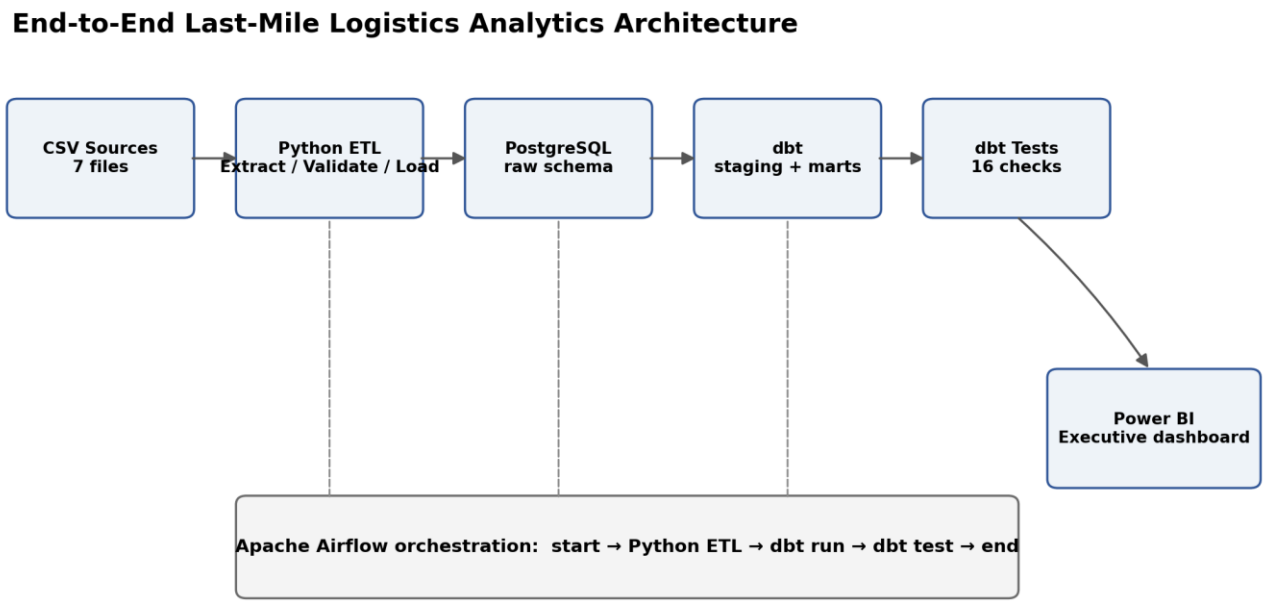


Figure 1. End-to-end architecture implemented in the project.

Python ETL: Reads CSV sources, validates structural quality, bulk-loads PostgreSQL raw tables and verifies row counts.

PostgreSQL: Acts as the persistent relational store for raw and analytics layers.

dbt: Applies transformation logic in modular SQL models and executes automated data-quality tests.

Apache Airflow: Controls dependency order so downstream tasks execute only after upstream tasks succeed.

Power BI: Consumes analytics-layer tables/views and presents operational performance to a business audience.

4. Python ETL Pipeline

The ETL script follows a five-part memory rule used throughout the project: Extract → Validate → Load → Verify → Log. The implementation is configuration-driven, with PostgreSQL credentials and the source-data location read from environment variables rather than hard-coded secrets.

4.1 Extract

Each expected CSV file is resolved from LOGISTICS_DATA_DIR and read into a pandas DataFrame. Missing source files cause the pipeline to fail immediately rather than silently producing partial data.

4.2 Validate

- Expected-column validation checks for both missing and unexpected columns.
- Primary-key validation rejects NULL and duplicate key values.
- Missing-value profiling records warnings for non-key fields without automatically discarding valid operational records.

4.3 Load

The raw PostgreSQL tables are truncated as a single refresh set, after which each CSV is loaded with PostgreSQL COPY through psycopg2. COPY was chosen to support efficient bulk ingestion at the scale of millions of tracking-event records.

4.4 Verify and Log

After each table is loaded, the database row count is compared with the source row count. Any mismatch raises an exception. Structured logs capture extraction, validation, profiling, load and verification events, making the workflow auditable and easier to troubleshoot.

5. PostgreSQL and Analytics Layering

PostgreSQL is used as the central database. The project keeps raw ingestion separate from transformation logic: raw tables preserve source-level records, while downstream models/views provide cleaner analytical structures. This separation avoids embedding business transformation rules inside the ingestion script.

Layering approach

Layer	Examples	Purpose
Raw	raw.orders, raw.tracking_events, raw.payments	Faithful landing zone for source data
Staging	stg_orders, stg_tracking_events	Standardization and reusable cleaning logic
Intermediate	int_order_events	Reusable business logic between staging and marts
Analytics / marts	fct_orders, mart_daily_kpi; analytics views	Business-ready facts, KPIs and dashboard inputs

This layered structure also makes debugging easier: when a KPI appears incorrect, the analyst can trace the result back through the mart, intermediate model, staging model and raw source.

6. dbt Transformations and Data Quality

dbt is used to manage SQL transformations as versionable models rather than one-off database queries. The recovered project includes staging models, an intermediate order-events model, an order fact model, a daily KPI mart, source definitions, model tests and a custom range test.

6.1 Transformation flow

- stg_orders and stg_tracking_events prepare source data for reuse.
- int_order_events consolidates event-level logic needed by downstream reporting.
- fct_orders creates an order-level analytical grain with delivery, SLA, return and operational indicators.
- mart_daily_kpi aggregates order activity into daily operational metrics.

6.2 Data-quality controls

The final dbt test run executed 16 tests. The suite covered not-null checks, uniqueness checks, accepted-value checks, source identifier checks and a custom KPI-range assertion. All 16 tests passed with zero warnings and zero errors.

7. Apache Airflow Orchestration

The project initially validated the Python ETL as a standalone Airflow task. It was then extended into a complete five-task DAG that coordinates ingestion, transformation and testing.

```
start
   ↓
run_python_etl
   ↓
run_dbt
   ↓
test_dbt
   ↓
end
```

This dependency structure ensures that dbt transformations are not executed when ingestion fails, and dbt tests do not run until the models have been built successfully. The final manual Airflow run completed successfully across all five tasks.

Current scheduling status: The DAG is configured for manual execution (schedule=None) during the portfolio testing phase. Automatic scheduling, retry policies and alerting are intentionally deferred to the next iteration.

8. Power BI Dashboard and Key Results

The final business-facing deliverable is a single-page Power BI dashboard designed for rapid operational review. It uses analytics-layer tables/views rather than rebuilding transformation logic inside Power BI.

Dashboard components

- Headline cards: Total Orders, SLA Compliance, Average Delivery TAT and Delivery Success Rate.
- Quarterly Order Volume trend.
- Order Status Distribution.
- Delivery Success Rate by Hub.
- Top Courier Performance.

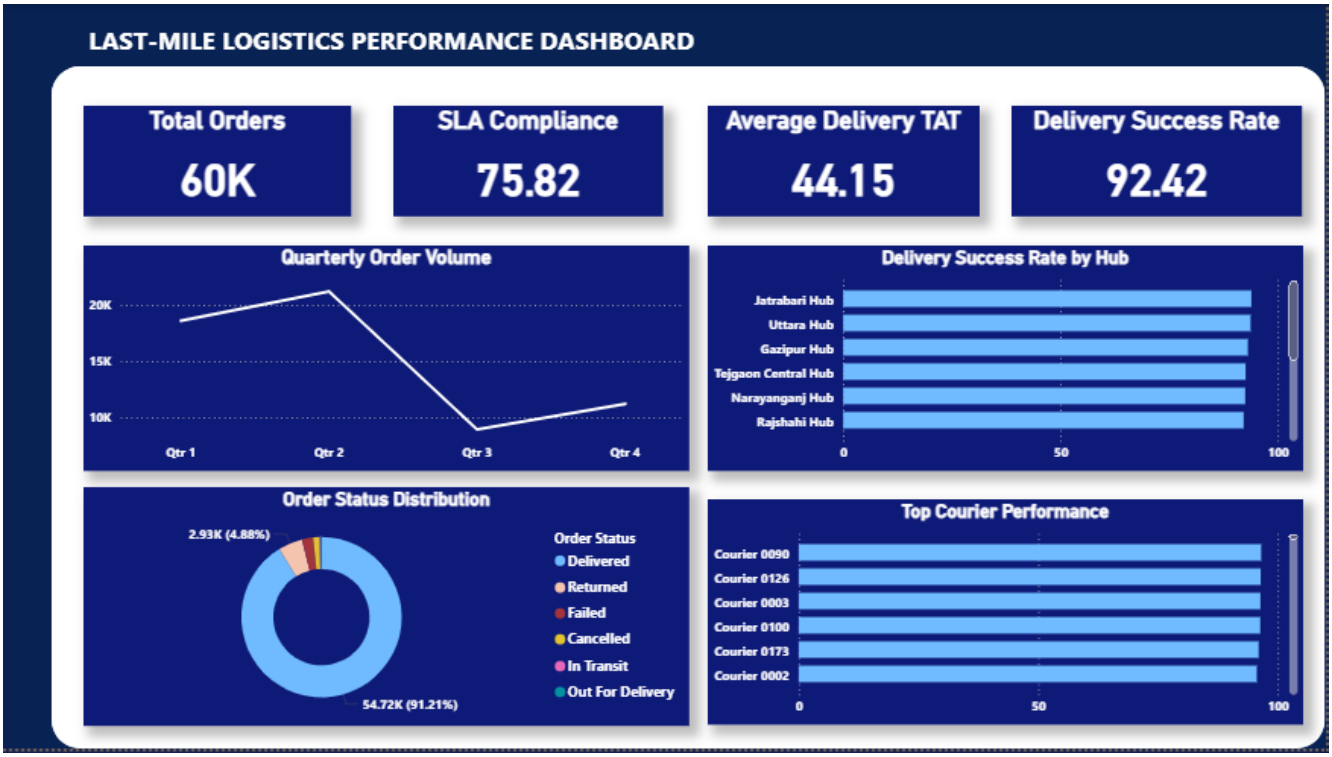


Figure 5. Final one-page Last-Mile Logistics Performance Dashboard in Power BI.

Headline metrics displayed in the final dashboard

Metric	Displayed result
Total Orders	**60K**
SLA Compliance	**75.82**
Average Delivery TAT	**44.15 hours**
Delivery Success Rate	**92.42**

The dashboard also shows that the delivered status dominates the order-outcome distribution. Because dashboard measures can use different eligible populations or filters, displayed rates are treated as model-defined KPIs rather than recomputed from unrelated visuals.

9. Validation, Reliability and Engineering Challenges

9.1 End-to-end validation evidence

- Every source file passed schema and primary-key ingestion checks.
- Source and PostgreSQL row counts matched for all seven raw tables.
- The largest table, tracking_events, loaded and verified at 4,244,332 rows.
- dbt completed 16 of 16 configured data-quality tests successfully.
- The final Airflow DAG completed successfully across all five tasks.
- Power BI connected to the PostgreSQL analytics layer and rendered the final KPI dashboard.

9.2 Engineering challenges encountered

A significant part of the project involved recovering from WSL filesystem and environment instability. Symptoms included input/output errors, read-only filesystem behavior, a damaged Python virtual environment, and difficulty using the original Ubuntu instance after moving WSL storage.

The recovery approach was deliberately conservative:

- Preserve recoverable project source files before rebuilding the runtime environment.
- Install a clean Ubuntu/WSL instance and verify basic filesystem writes and swap behavior.
- Restore only source code, data and configuration templates rather than reusing the damaged virtual environment.
- Recreate Python and Airflow virtual environments from clean packages.
- Reconnect PostgreSQL through environment variables and revalidate the ETL before moving to dbt and Airflow.
- Run each stage independently before restoring end-to-end orchestration.

This recovery sequence reduced the risk of carrying corrupted binaries or generated files into the rebuilt environment and reinforced the value of separating source code from runtime artifacts.

10. Limitations and Future Roadmap

The current version is complete for portfolio submission, but several engineering capabilities are intentionally outside the present scope.

Area	Current state	Recommended next step
Scheduling	Manual Airflow trigger	Add daily/interval schedule with timezone-aware execution
Resilience	Basic task failure propagation	Add retries, retry delays and task-level timeouts
Observability	Console/file logs	Add alerting and structured run monitoring
dbt scalability	Full-refresh portfolio models	Introduce incremental models where justified
Deployment	Local WSL + Windows PostgreSQL	Containerize services and document

		reproducible startup
CI/CD	Not implemented	Run dbt tests and code checks on pull requests
BI depth	Single executive page	Add slicers, drill-through, merchant/COD/return analysis and data-quality pages

11. Skills Demonstrated and Conclusion

Technical skills demonstrated

- Python data engineering with pandas, psycopg2, logging and environment-variable configuration.
- PostgreSQL bulk loading, relational layering and analytical SQL workflows.
- dbt model organization, source definitions, tests and custom assertions.
- Apache Airflow DAG development and dependency-based orchestration.
- Power BI executive dashboard construction and KPI communication.
- Linux/WSL environment management, virtual environments and troubleshooting.
- Git/GitHub repository hygiene including secret exclusion through .gitignore and .env.example.

Business and analytical skills demonstrated

- Translation of operational logistics data into measurable delivery KPIs.
- Separation of technical data layers from business-facing analytics.
- Data-quality thinking across schema, identifiers, row counts and metric ranges.
- Clear presentation of delivery performance by status, time, hub and courier.

Conclusion. The project successfully demonstrates a complete analytics lifecycle: source ingestion, validation, relational storage, SQL transformation, automated testing, workflow orchestration and business visualization. The final implementation is suitable as a portfolio artifact because the main components were not only designed but executed and verified together. The next phase should focus on production hardening rather than adding more core components.

Appendix A. Reproducible Run Sequence

The following sequence summarises the operational flow used in the final environment. Exact credentials are intentionally excluded.

Activate the project environment

```
cd ~/logistics_project
source project_venv/bin/activate
source .env
```

Run Python ETL

```
python python_etl/python_etl.py
```

Run dbt

```
cd logistics_dbt_starter
dbt debug --profiles-dir .
dbt run --profiles-dir .
dbt test --profiles-dir .
```

Start / test Airflow

```
export AIRFLOW_HOME="$HOME/airflow"
source ~/airflow/airflow_venv/bin/activate
airflow dags list-import-errors -l
airflow standalone
```

Security note: Real PostgreSQL credentials are kept in the local .env file. The repository template uses .env.example and dbt environment-variable references so secrets are not committed to version control.

Appendix B. Repository Structure

```
logistics_project/
├── airflow/
│   └── dags/
│       └── run_logistics_etl.py
├── data/
│   └── carrybee_logistics_dataset/
├── python_etl/
│   └── python_etl.py
├── logistics_dbt_starter/
│   ├── models/
│   │   ├── staging/
│   │   ├── intermediate/
│   │   └── marts/
│   ├── tests/
│   ├── dbt_project.yml
│   └── profiles.yml
├── .env.example
├── .gitignore
└── README.md
```

Excluded runtime artifacts include .env, Python virtual environments, dbt target/log directories, Python cache files and generated logs.